Build a cluster and a platform control plane

Read the context below and complete the challenge. See **Evaluation** for how we will evaluate the result.

Context

As a P4 in Kiratech, you will work as part of a project team led by a senior engineer, on systems which are often already in production. You will receive a design that has been agreed with the client, and you will be responsible for implementing your part of it reliably, documenting what you did and raising problems early. Many of our engagements start from the foundations: installing and operating Kubernetes clusters on the client's infrastructure, and building on top of them a control plane that lets teams consume the platform in self-service.

We also expect our engineers to work effectively with AI assistants, and to stay fully responsible for what they produce with them.

To this end, the challenge for Cloud Native Engineer candidates at P4 level is to demonstrate that they understand how a Kubernetes cluster works below `kubectl apply`, that they can operate its control plane safely, and that they can expose platform capabilities through a declarative API. Below, we have constructed a scenario which typically confronts a P4 Engineer in Kiratech. Considering the client requirements and the decisions already made, respond to the challenge below.

Client requirements

The client is modernising their data centre operations to adopt greater automation, with the aim of eventually creating an internal developer platform. Their data centre runs on VMWare and consists of three sites; the south site hosts non-production workloads.

The client wants to replace a set of manually installed, single-master Kubernetes clusters with clusters that are **highly available, reproducible and upgradable** without downtime for the workloads. On top of them, business teams must be able to request a **team environment** in self-service, instead of opening a ticket to the Platform Engineering team.

Decisions already made

The project lead has agreed the following with the client. You do not need to question them, but you can point out risks or improvements if you see any.

- Clusters run on virtual machines. Each cluster has **3 control-plane nodes** and at least **1 worker node**, and the Kubernetes API is reachable through a single stable endpoint (load balancer or virtual IP).
- The distribution is your choice among **kubeadm, RKE2 and k3s** (in HA mode); explain why.
- Everything is defined as code and stored in Git: cluster installation, configuration and platform components.
- A team environment consists of: a namespace, a ResourceQuota and a LimitRange, RBAC for the team's group, a default-deny NetworkPolicy, and an Argo CD Application pointing to the team's repository.
- The self-service API is implemented with one of the following options:
  - **Option A - Krateo PlatformOps** (open source, born in Kiratech): the team environment is packaged as a Helm chart with a `values.schema.json` and registered as a `CompositionDefinition`, so that the platform exposes a new Kubernetes API for team environments.
  - **Option B - an alternative of your choice** (e.g. Argo CD ApplicationSet, Crossplane, or another tool), as long as a team environment can be requested through a declarative resource stored in Git.

  Both options are valid. Choose the one you can deliver best, and explain your choice.

Challenge

You are a member of the project team. Build a reference cluster and the first self-service capability on top of it.

Part 1 - Build the cluster

Create a Kubernetes cluster with 3 control-plane nodes and at least 1 worker on local virtual machines (e.g. Multipass, Vagrant, Lima, Proxmox) or on a cloud free tier. The installation must be scripted end to end (e.g. Ansible, Terraform, shell scripts) and documented. Explain:

- how the API endpoint is made highly available (e.g. HAProxy + keepalived, kube-vip);
- where etcd runs (stacked or external) and why;
- which CNI you chose and why.

If you cannot run virtual machines, you can use a multi-node kind cluster with 3 control-plane nodes, but you must explain what would be different on real VMs.

Part 2 - Operate the control plane

Show, with commands and outputs, that you can operate the cluster safely:

a. **Failure tolerance** - stop one control-plane node and show that the API still works. Then explain (you do not need to do it) what happens if a second control-plane node is lost, and why.
b. **Backup and restore** - take an etcd snapshot, delete a namespace with some resources in it, restore the snapshot and show that the namespace is back. Explain the risks of a restore on a cluster in production.
c. **Upgrade** - upgrade the cluster by one minor version of Kubernetes without interrupting a sample workload (e.g. an nginx Deployment with 2 replicas probed every second). Describe the order of operations and the version skew rules you followed.

Part 3 - Build the platform control plane

Implement the **team environment** capability with Option A or Option B.

- A platform engineer must be able to create a new team environment by committing a single declarative resource to Git (or applying it with `kubectl`), with a few validated parameters (e.g. team name, size `S/M/L`, team repository URL).
- Invalid requests (e.g. unknown size, missing team name) must be rejected by the API, not discovered later.
- Deleting the resource must remove the environment.
- A workload deployed in a team environment must work under the default-deny NetworkPolicy: allow only the flows it actually needs.

The platform control plane can run on the cluster built in Part 1 or on a separate local cluster (e.g. kind) acting as a management cluster. Krateo has significant resource requirements: if you choose Option A and your machine is limited, a separate kind cluster is acceptable. In that case, explain how the management cluster would deploy environments on the cluster built in Part 1.

Part 4 - Work with AI assistants

You are free, and encouraged, to use AI assistants during the challenge (e.g. Claude, ChatGPT, GitHub Copilot, Cursor) and AI tools for Kubernetes operations (e.g. k8sgpt). You remain responsible for every line of code and every command you run. Write a short note (max 1 page) that explains:

- where you used AI assistants and where you deliberately did not, and why;
- at least one case in which the assistant gave you an answer that was wrong, outdated or unsafe for a production cluster, and how you found out;
- how you verified generated code, manifests and commands before running them on the cluster (e.g. dry-run, documentation, tests, a disposable cluster).

If you did not use any AI assistant, explain how you would introduce one safely in the team's day-to-day work on this platform.

Part 5 - Troubleshoot

For each of the four situations below, explain the most likely cause, how you would confirm it, and how you would fix it. Where more than one cause is possible, say which ones and how you would tell them apart.

Situation 1. On a cluster installed over a year ago:

```
$ kubectl get nodes
NAME       STATUS     ROLES           AGE    VERSION
cp-1       Ready      control-plane   398d   v1.33.4
cp-2       Ready      control-plane   398d   v1.33.4
cp-3       Ready      control-plane   398d   v1.33.4
worker-1   NotReady   <none>          398d   v1.33.4

worker-1$ journalctl -u kubelet | tail -1
... "Unable to authenticate the request" err="[x509: certificate has
expired or is not yet valid: current time 2026-10-12T09:14:02Z is after
2026-10-10T08:02:11Z]"
```

Situation 2. A colleague added a fourth control-plane node to test a procedure, then deleted its VM. Since then the team has noticed that the cluster "no longer tolerates the loss of a control-plane node":

```
$ etcdctl member list -w table
+------------------+---------+------+---------------------------+
|        ID        | STATUS  | NAME |        PEER ADDRS         |
+------------------+---------+------+---------------------------+
| 2a1c...          | started | cp-1 | https://10.0.10.11:2380   |
| 7f3b...          | started | cp-2 | https://10.0.10.12:2380   |
| 9e05...          | started | cp-3 | https://10.0.10.13:2380   |
| c41d...          | started | cp-4 | https://10.0.10.14:2380   |
+------------------+---------+------+---------------------------+
```

Situation 3. A new Deployment stays Pending:

```
$ kubectl describe pod web-6d4f9c7b8-kx2lp | tail -2
Warning  FailedScheduling  0/4 nodes are available: 1 Insufficient memory,
3 node(s) had untolerated taint {node-role.kubernetes.io/control-plane: }.
```

A colleague proposes to remove the taint from the control-plane nodes. What do you answer?

Situation 4. A team has just received its environment from the self-service API and deployed its application. The pods are Running, but the application logs show that it cannot connect to its database, which runs in the same namespace:

```
$ kubectl -n team-blue exec deploy/api -- nslookup db
;; connection timed out; no servers could be reached
```

```
$ kubectl -n team-blue exec deploy/api -- nc -zv 10.96.41.7 5432
10.96.41.7 (10.96.41.7:5432): Connection timed out
```

The team asks you to "just delete the NetworkPolicy". What do you answer, and what do you change in the team environment template so that the next team does not hit the same problem?

Definition of Done

1. A **Git repository** with all the code, and a README that allows a colleague to recreate the cluster and the platform control plane from scratch, with the exact commands.
2. **Two C4 diagrams**, provided both as code (e.g. Structurizr DSL, PlantUML, Mermaid) and as rendered images:
   a. a Container diagram of the platform control plane and of the clusters it manages (Git, Argo CD, Krateo or alternative, workload cluster);
   b. a Deployment diagram showing VMs, control-plane and worker nodes, the API endpoint, etcd and the CNI.
3. **Four short decision records** (ADR format, half a page each is enough): the distribution, the etcd and API endpoint topology, Option A vs Option B, and the structure of the Git repositories (what is reconciled by whom).
4. **Evidence for Part 2 and Part 3** (commands and outputs, or a short screen recording).
5. The **note on AI assistants** (Part 4).
6. The **answers to Part 5**.

Extra Credit

- Install the cluster with GitOps from day one: after bootstrap, all platform components (CNI excluded) are reconciled by Argo CD.
- With Option A, expose the team environment in the Krateo portal with a form, so that a non-technical user can request it.
- Automate the etcd backup (e.g. a CronJob that ships snapshots to an object store) and write a short restore runbook.
- Register the cluster built in Part 1 as a target of the management cluster, and create a team environment on it from the management cluster.
- Show how an AI tool for Kubernetes operations (e.g. k8sgpt) could help diagnose one of the situations in Part 5, and where its answer would not be enough.

Evaluation

You will be evaluated based on the quality and reproducibility of what you deliver, your understanding of how the control plane works, your ability to reason logically and explain your choices, and the clarity of your documentation. A smaller solution that works and is

well explained is worth more than a large one that is incomplete. If you make assumptions or skip something, say so explicitly.

We will discuss your solution with you in a follow-up session, where you will be asked to show it running and to perform a small operation live (e.g. adding a team environment with a different size or replacing a control-plane node).

English or Italian can be used.